INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

COURSE TITLE: BACHELOR OF TECHNOLOGY

SUBJECT: PYTHON PROGRAMMING LABORATORY

PROJECT TITLE: SENSOR CALIBRATION COMPARISON

NAME: 1. **RUDRANIL MUKHERJEE** 2. **BAIBHAV JANA**

3. **MD. IMRAN RIZWI** 4. **VIRAJ KISHOR**

ROLL NO: **130, 165, 164, 161**

ENROLLMENT NO: **12025052003140, 12025052003177, 12025052003176, 12025052003173**

SECTION: **B**

SUBMITTED TO: **PYTHON LABORATORY**

DEPARTMENT NAME: ELECTRONICS AND COMMUNICATION ENGINEERING

ACADEMIC SESSION: 2026-2027

SENSOR CALIBRATION COMPARISON

Abstract

This project develops a Python workflow to compare raw sensor readings with known reference values and evaluate whether linear calibration improves measurement accuracy. It validates paired records, fits a correction using training observations, evaluates held-out data, exports CSV/JSON summaries and generates labelled plots. The implementation uses Python, a CalibrationRun class, NumPy, Pandas and Matplotlib. The demonstration dataset is synthetic, not collected from a physical sensor. In the initial run, held-out MAE decreased from 8.492 to 1.106 units (about 87.0%). These results demonstrate the workflow, not real-sensor performance.

Keywords

Sensor calibration; Python; CSV; NumPy; Pandas; Matplotlib; linear regression; MAE; RMSE.

Table of Contents

- 1. Introduction
- 2. Problem Statement and Objectives
- 3. Software and System Requirements
- 4. System Design and Methodology
- 5. Implementation of Major Modules
- 6. Results and Discussion
- 7. Testing and Validation
- 8. Limitations and Future Scope
- 9. Conclusion
- 10. References
- 11. Appendix

SENSOR CALIBRATION COMPARISON

1. Introduction

Electronic sensors may report values that differ from a trusted reference because of offset, scale error and noise. Calibration estimates a relationship between sensor output and known reference values so readings can be corrected. This Python application loads paired data, validates it, fits a linear model and visualises performance.

2. Problem Statement and Objectives

Problem: compare raw measurements with known references, estimate a linear correction from paired observations, and test whether it reduces error on data excluded from fitting.

- Validate paired records and reject missing or invalid values.
- Represent observations using a CalibrationRun class and CSV records.
- Fit a linear calibration equation with NumPy using training data only.
- Calculate MAE, RMSE and mean error before and after correction.
- Use Pandas for summaries and CSV/JSON export.
- Generate two labelled Matplotlib plots and automated tests.

3. Software and System Requirements

Component	Purpose
Python 3.9+	Program execution
NumPy	Linear fitting and error metrics
Pandas	Tabular validation and summaries
Matplotlib	Scatter plot and error histogram
CSV / JSON	Input and output data
unittest	Automated testing

SYSTEM DESIGN AND IMPLEMENTATION

4. System Design and Methodology

4.1 Dataset schema

Each CSV row contains run_id, reference_value, raw_reading and split. The split field identifies training or held-out test observations.

4.2 Data preparation

The generation script uses a fixed random seed to create 44 synthetic paired observations across reference levels from 0 to 100 units. Raw values include scale error, offset and random noise.

4.3 Workflow

- Load CSV and validate required columns, missing values, ranges and split labels.
- Separate training and test records.
- Fit reference = slope × raw + intercept using training data.
- Apply correction and calculate held-out raw/corrected metrics.
- Export calibrated CSV and JSON summary; generate plots.

4.4 Metrics

MAE is the mean absolute difference from the reference. RMSE is the square root of mean squared error and weights larger errors more strongly. Mean error indicates signed bias.

5. Implementation of Major Modules

Module	Function
CalibrationRun	Stores and validates one observation
load_data()	Loads and validates CSV
fit_calibration()	Fits linear correction using NumPy
error_metrics()	Computes MAE, RMSE and mean error
analyse()	Applies correction and evaluates test data
make_plots()	Creates labelled visualisations
main()	Exports reports and runs workflow

RESULTS AND DISCUSSION

6. Results and Discussion

6.1 Results

Indicator	Observed value
Input observations	44
Training observations	33
Held-out test observations	11
Reference range	0–100 units
Raw test MAE	8.492 units
Corrected test MAE	1.106 units
MAE reduction	87.0%

The correction reduced average absolute error on the held-out synthetic observations. Test records were not used to fit the equation, providing a basic check of generalisation within this generated sample.

6.2 Calibration scatter plot

Figure 1. Held-out raw and corrected readings compared with the ideal reference line.

6.3 Absolute-error histogram

Figure 2. Distribution of held-out absolute errors before and after calibration.

The scatter plot shows agreement with reference values; the histogram compares error magnitudes. Numerical metrics quantify the change.

TESTING, CONCLUSION AND APPENDIX

7. Testing and Validation

Test	Expected behaviour	Status
Valid record	Accepted	Passed
Negative reading	Rejected	Passed
Linear fit	Correct slope/intercept	Passed
Error metrics	Correct MAE	Passed
Train/test split	Both present	Passed
Calibration evaluation	Test MAE decreases	Passed

Run tests with: python -m unittest discover -s tests -v

8. Limitations and Future Scope

- Dataset is synthetic and cannot validate a physical sensor.
- Linear correction assumes an approximately linear sensor response.
- Small samples may produce split-sensitive results.
- Future work: collect real Arduino sensor readings against a traceable reference, test nonlinear correction and evaluate drift.

9. Conclusion

The project demonstrates a reproducible Python calibration workflow using validation, object-oriented records, NumPy, Pandas, Matplotlib and automated tests. The synthetic held-out sample showed an approximately 87% MAE reduction. Real measurements are required for practical validation.

10. References

- Python Documentation: https://docs.python.org/3/
- NumPy Documentation: https://numpy.org/doc/
- Pandas Documentation: https://pandas.pydata.org/docs/
- Matplotlib Documentation: https://matplotlib.org/stable/

11. Appendix: Dataset and Execution

CSV headers: run_id, reference_value, raw_reading, split.

Install: python -m pip install numpy pandas matplotlib

Run: python main.py | Regenerate: python generate_dataset.py | Tests: python -m unittest discover -s tests -v

Individual Contributions

Complete contributions.md with actual team members, roll numbers and verified contributions.